Why FP in Java? · my notes L01

One rule all lesson long: "a transaction is high value when amount > 1,000.00"

1 · HOW vs WHAT

HOW (imperative)

I write the **loop**, the **if**, and a **mutable total**.

```
for… if… total = total.add(…)
```

Java 8

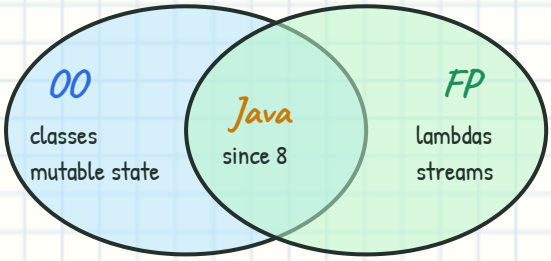
WHAT (declarative)

I state the **recipe**: filter debits → amounts → add up.

```
.filter .map .reduce
```

Both print `16692.44` → same answer, but WHAT reads like the requirement. A loop still runs, the library owns it.

2 · Java = OO + FP features



NOT pure FP like Haskell:

- data is mutable by default
- side effects allowed anywhere
- no tail-call optimisation

→ functional style is a discipline I choose.

3 · Behaviour parameterisation = the code shrinks

Strategy class
~13 lines
+ a new file per rule
1 line is real logic!

→

Anonymous class
6 lines
inline, still ceremony

→

Lambda
1 line ✓

`t -> t.amount().compareTo(THRESHOLD) > 0` works with the OLD **TransactionFilter** because it has exactly 1 abstract method. All steps → the same **11** transactions.

4 · Four core ideas

Functions as values
store · pass · return · compose
`discountOf(10).andThen(ADD_GST)`

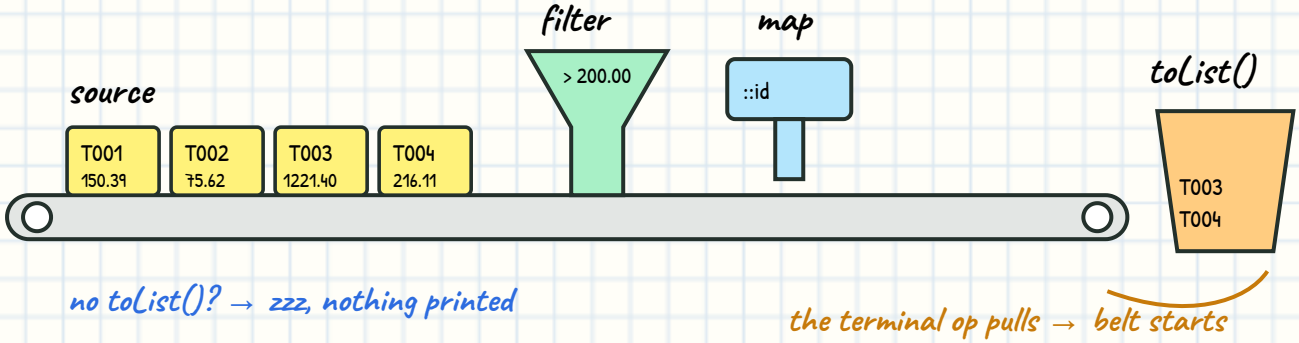
Immutability
records [16] · List.of [9]
withdraw → a NEW Account

Pure functions
same in → same out, no side effects
`fee(2500.00) = 12.50` always

Laziness
nothing runs until a terminal op asks (→ page 2)

5 · Laziness = a conveyor belt

nothing moves until someone at the end **pulls** (the terminal operation)



no toList()? → zzz, nothing printed

the terminal op pulls → belt starts

peek() trace

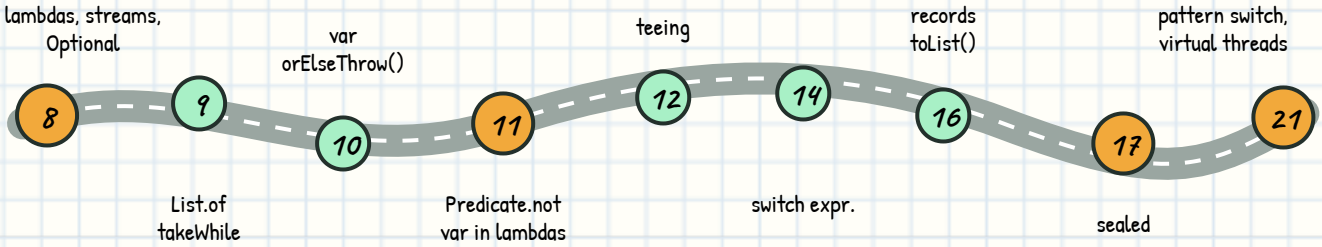
filter T001 ×
filter T002 ×
filter T003 → **map T003**
filter T004 → **map T004**
= [T003, T004]

vertical, not horizontal: each element goes through ALL steps before the next one starts (loop fusion).

short-circuit: `limit(2)` found 2 high-value txns after looking at only **19 of 60**.

peek = debugging only, never business logic.

6 · The road from Java 8 to 21



orange = LTS (8 · 11 · 17 · 21). `beyond 21:` unnamed variables _ (22), Stream Gatherers (24).

7 · Pure vs impure

pure: 12.50, 12.50
always the same

impure: 12.50 … rate changed … 25.00
same input?!

★ Remember

- `toList()` [16] → **unmodifiable**
- money = BigDecimal, scale 2, HALF_UP
- don't fill outside lists in `forEach`
- streams are a tool, not a religion
- next → L02 functional interfaces